

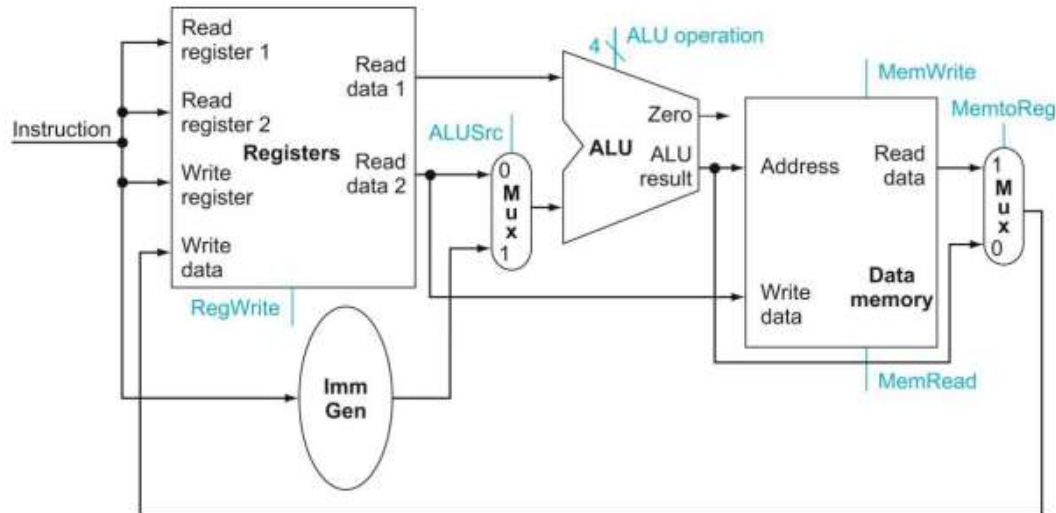
Assignment 05 (Solution)

Exercise 1:

Consider the following instruction:

Instruction : and rd, rs1, rs2

Interpretation: $\text{Reg}[\text{rd}] = \text{Reg}[\text{rs1}] \text{ AND } \text{Reg}[\text{rs2}]$



- 1) What are the values of control signals generated by the control in Figure 1 for this instruction?

Regwrite=1 (True)

ALUSrc= 0

ALU operation = and

MemRead = 0 (false)

MemWrite = 0 (false)

MemtoReg = 0

Mathematically, the MemRead control signal is a “don’t care”: the instruction will run correctly regardless of the chosen value. Practically, however, MemRead should be set to false to prevent causing a segment fault or cache miss.

- 2) Which resources (blocks) perform a useful function for this instruction?

Registers, ALUSrc mux, ALU, and the MemToReg mux

- 3) Which resources (blocks) produce no output for this instruction?

All blocks produce some output. The outputs of DataMemory and Imm Gen are not used.

- 4) Which resources produce output that is not used? 4. Explain each of the “don’t cares” in the following table

Instruction	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

MemToReg = X for sw and beq : Neither sw nor beq write a value to the register file.
It doesn't matter which value the MemToReg mux passes to the register file because the register file ignores that value

- 5) When silicon chips are fabricated, defects in materials (e.g., silicon) and manufacturing errors can result in defective circuits. A very common defect is for one signal wire to get "broken" and always register a logical 0. This is often called a "stuck-at-0" fault.
 - a. Which instructions fail to operate correctly if the MemToReg wire is stuck at 0?
Only loads are broken. MemToReg is either 1 or "don't care" for all other instructions
 - b. Which instructions fail to operate correctly if the ALUSrc wire is stuck at 0?
I-type, loads, stores are all broken.

Exercise 2:

Given the single-cycle CPU datapath in Figure 2

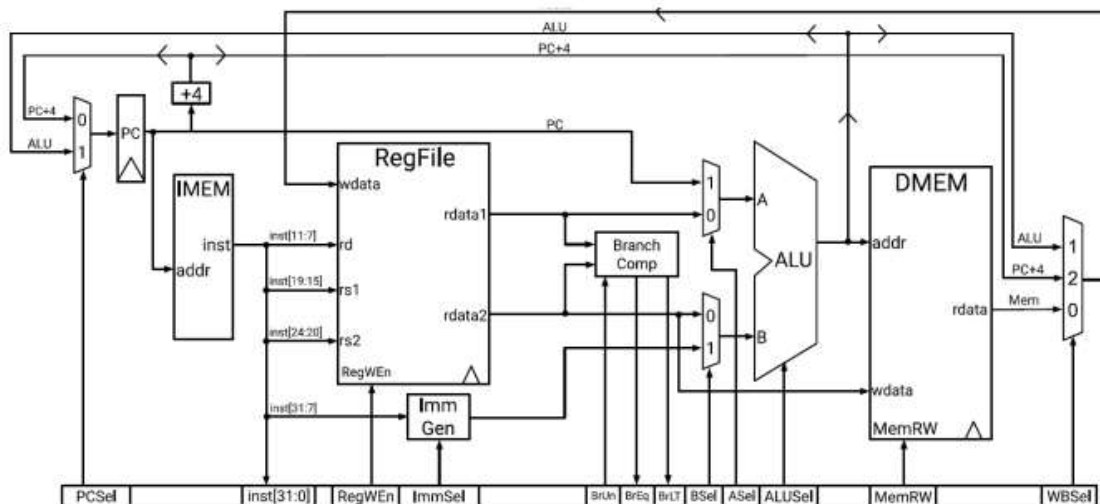


Figure 2: Single cycle datapath

- 1) List all possible values that each control signal may take on for the single cycle datapath, then briefly describe what each value means for each signal.
 - a. PCSel
 - 0: PC = Old PC + 4
 - 1: PC = ALU output (PC+ imm for branches, jumps, etc. . .)

- b. RegWEn
0: WB value not written to register in RegFile
1: WB value is written
- c. ImmSel (3bits)
0-4: used for I, B, S, J, U-type immediates.
5-7: unused
- d. BrEq
0: set to 0 when rdata1 is not equal to rdata2
1: set to 1 when rdata1 is equal to rdata2
- e. BrLt
0: set to 0 when rdata1 is not less than rdata2
1: set to 1 when rdata1 is less than rdata2
- f. ALUSel (4bits)
0: add (0000)
1: sub (0001)
2: and (0010)
3: or (0011)
...etc.
- g. MemRW
0: disables writing to memory (Read)
1: enables writing to memory
- h. WBSel
0: DMEM read output
1: ALU output
2: PC + 4
3: unused

2) Fill out the following table with the control signals for each instruction based on the datapath

	BrEq	BrLT	PCSel	Imm-Sel	BrUn	ASel	BSel	ALUSel	MemRW	Reg-WEn	WB-Sel
add	*	*	0 (PC + 4)	*	*	0 (Reg)	0 (Reg)	add	0	1	1 (ALU)
ori	*	*	0	I	*	0 (Reg)	1 (Imm)	or	0	1	1 (ALU)
lw	*	*	0	I	*	0 (Reg)	1 (Imm)	add	0	1	0 (MEM)
sw	*	*	0	S	*	0 (Reg)	1 (Imm)	add	1	0	*
beq	0/1	*	0/1	B	*	1 (PC)	1 (Imm)	add	0	0	*
jal	*	*	1 (ALU)	J	*	1 (PC)	1 (Imm)	add	0	1	2 (PC + 4)
blt	*	0/1	0/1	B	0	1 (PC)	1 (Imm)	add	0	0	*

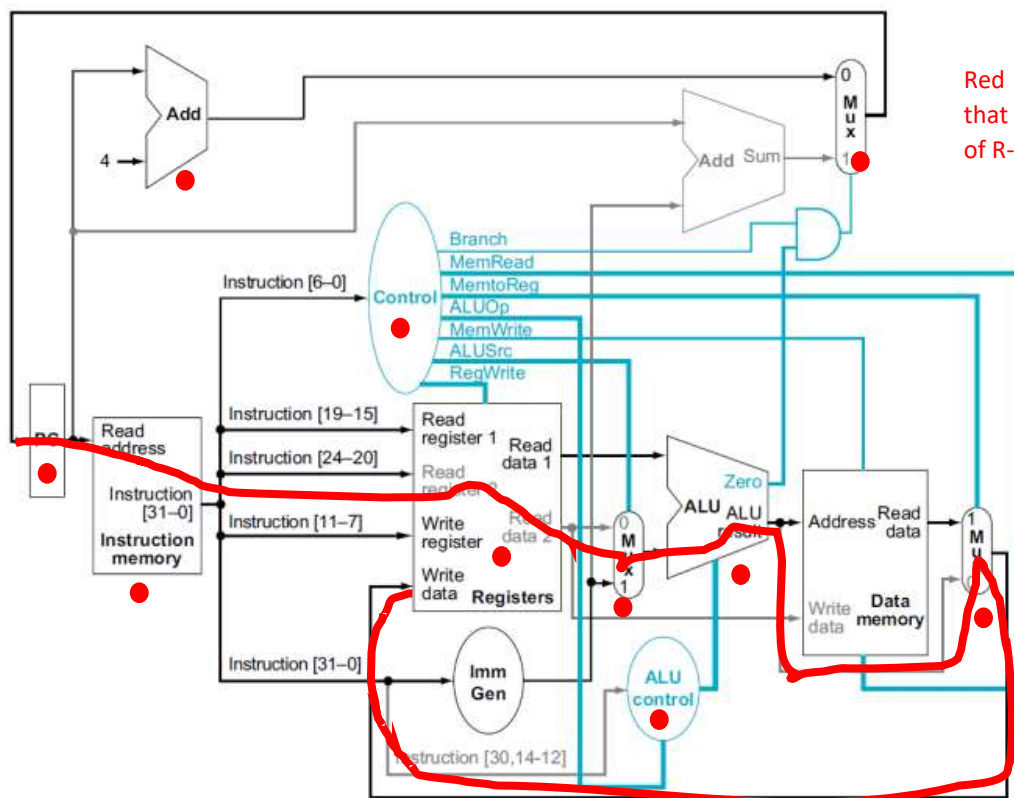
Problems in this exercise assume that the logic blocks used to implement a processor's datapath (Figure 3) have the following latencies:

I-Mem / D-Mem	Register File	Mux	ALU	Adder	Single gate	Register Read	Register Setup	Sign extend	Control
250 ps	150 ps	25 ps	200 ps	150 ps	5 ps	30 ps	20 ps	50 ps	50 ps

“Register read” is the time needed after the rising clock edge for the new register value to appear on the output. This value applies to the PC only. “Register setup” is the amount of time a register’s data input must be stable before the rising edge of the clock. This value applies to both the PC and Register File.

1. What is the latency of an R-type instruction, lw, sw, beq, and an arithmetic, logical, or shift I-type (non-load) instruction?

Latency of R-Type : Rop rd, rs1,rs2



Red circle represents circuits that intervene in the execution of R-type instruction

Execution path of R-Type instructions : In general, for each stage, we have several paths that execute in parallel (For timing calculation, we choose the longest path). For example:

- IF stage : add and IMEM execute in parallel.
- ID stage : RegFile, immGen and Controls circuits execute in parallel.
- Ex stage : (Alusrc Mux + Alu) and add execute in parallel

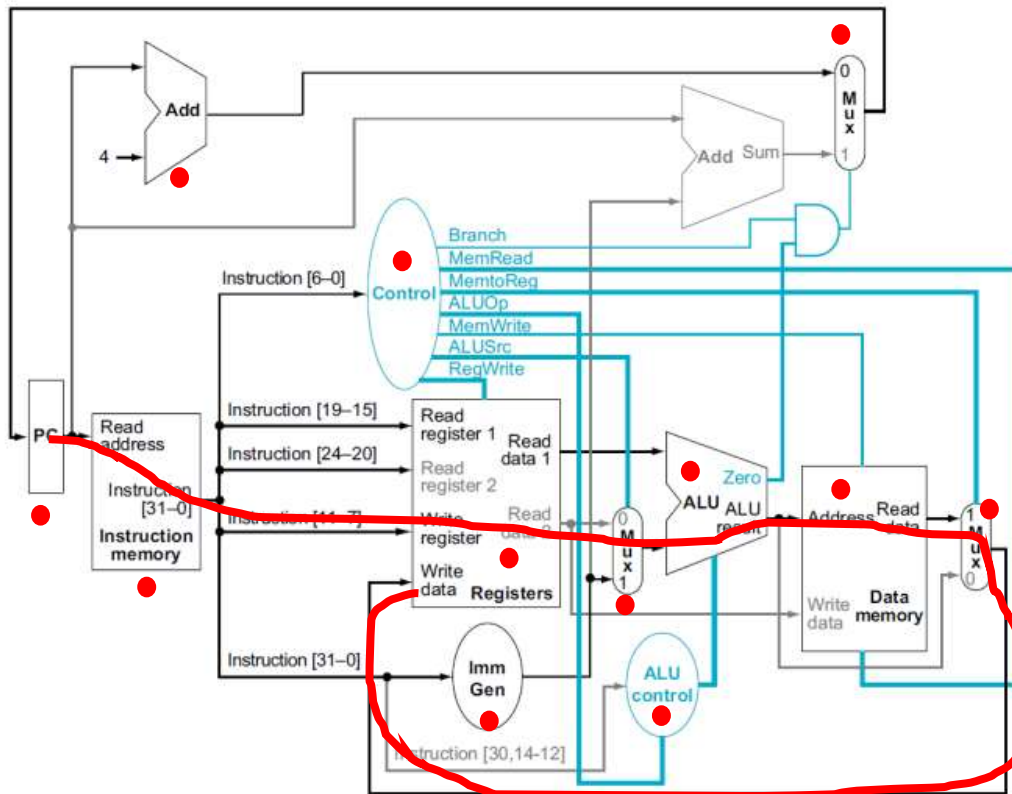
IMEM and Adder execute in parallel, therefore the execution time is the time of the longest path (In this case T_{IMEM})

$$T_R = T_{PC} + \max(T_{IMEM}, T_{adder}) + \max(T_{RegFile}, T_{ImmGen}, T_{control}) + T_{Mux} + T_{Alu} + T_{mux} + T_{setup}$$

$$T_R = 30 + 250 + 150 + 25 + 200 + 25 + 20 = 700ps$$

T_{setup} represents the time needed to make WB value stable before writing in RegFile (rd)

Latency of Lw : Lw rd, imm(rs1)



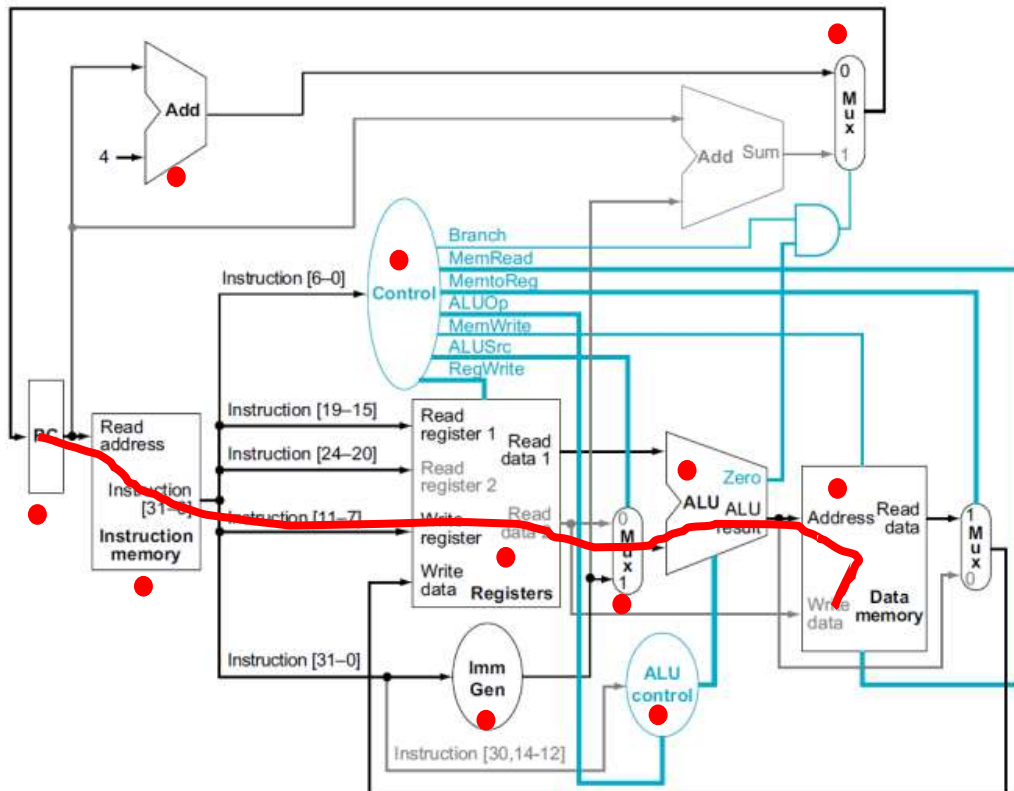
$$T_{Lw} = T_{PC} + \max(T_{IMEM}, T_{adder}) + \max(T_{RegFile}, T_{ImmGen}, T_{control}) + T_{Mux} + T_{Alu} + T_{IMEM} + T_{mux} + T_{setup}$$

$$T_{Lw} = 30 + 250 + 150 + 25 + 200 + 250 + 25 + 20 = 950ps$$

Latency of Sw : Sw rs2, imm(rs1)

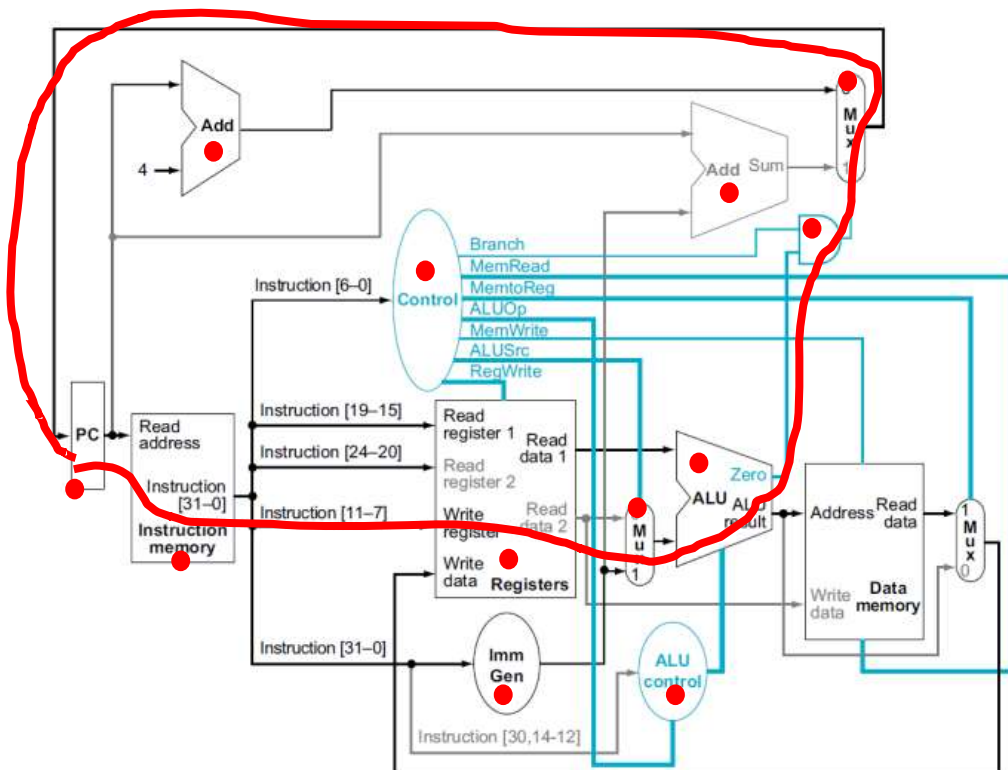
$$T_{Sw} = T_{PC} + \max(T_{IMEM}, T_{adder}) + \max(T_{RegFile}, T_{ImmGen}, T_{control}) + T_{Mux} + T_{Alu} + T_{IMEM}$$

$$T_{Sw} = 30 + 250 + 150 + 25 + 200 + 250 = 905ps$$



Latency of beq : beq rs1, rs2, label

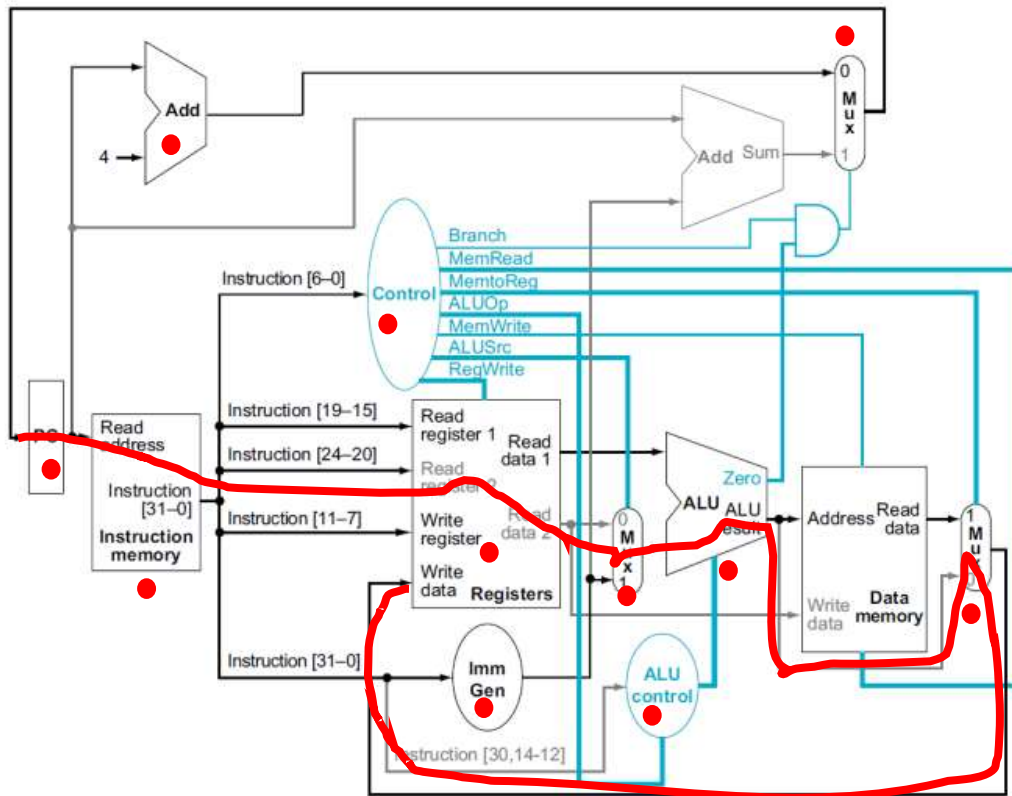
Here, we use Alu to compute zero output : $\text{zero} = \text{rs1} - \text{rs2}$ (if zero = 1 (true) : $\text{rs1} = \text{rs2}$ otherwise $\text{rs1} \neq \text{rs2}$). And we use adder to compute label : $\text{PC} + \text{imm}$. ($\text{PCsel} = \text{Branch and zero}$).



$$T_{\text{beq}} = T_{\text{PC}} + \max(T_{\text{IMEM}}, T_{\text{adder}}) + \max(T_{\text{RegFile}}, T_{\text{ImmGen}}, T_{\text{control}}) + T_{\text{Mux}} + T_{\text{Alu}} + T_{\text{gate}} + T_{\text{Mux}} + T_{\text{setup}}$$

$$T_{\text{Sw}} = 30 + 250 + 150 + 25 + 200 + 5 + 25 + 20 = 705 \text{ ps}$$

Latency of I-Type : Iop rd, rs1,imm



$$T_I = T_{\text{PC}} + \max(T_{\text{IMEM}}, T_{\text{adder}}) + \max(T_{\text{RegFile}}, T_{\text{ImmGen}}, T_{\text{control}}) + T_{\text{Mux}} + T_{\text{Alu}} + T_{\text{mux}} + T_{\text{setup}}$$

$$T_I = 30 + 250 + 150 + 25 + 200 + 25 + 20 = 700\text{ps}$$

Exercise 4:

Examine the difficulty of adding a proposed **lwi.d rd, rs1, rs2** ("Load With Increment") instruction to RISC-V. Interpretation: **Reg[rd]=Mem[Reg[rs1]+Reg[rs2]]**

- Which new functional blocks (if any) do we need for this instruction?
No new functional blocks are needed
- Which existing functional blocks (if any) require modification?
Only the control unit needs modification.
- Which new data paths (if any) do we need for this instruction?
No new data paths are needed.
- What new signals do we need (if any) from the control unit to support this instruction?
No new signals are needed.

